

Shiny para Python: : GUÍA RÁPIDA



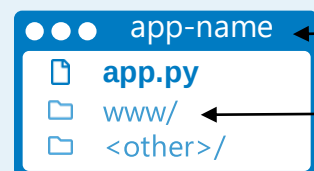
Crear una aplicación

Una aplicación Shiny es una página web interactiva (ui) impulsada por una sesión de Python en vivo ejecutada por un servidor (o por un navegador con Shinylive).



Los usuarios pueden manipular la interfaz de usuario, lo que hará que el servidor actualice las pantallas de la interfaz de usuario (mediante la ejecución de código Python).

Guarde la aplicación como app.py en un directorio con los archivos que usa.



El nombre del directorio es el nombre de la aplicación.

Coloque imágenes, CSS, etc. para compartir con el navegador en una carpeta llamada "www".

Funciones anidadas de Python para crear una interfaz HTML

Ejecute `shiny create .` en el terminal para generar una plantilla app.py archivo

Inicie aplicaciones con `shiny run app.py --reload`

Agregar entradas con funciones `ui.input_*`()

Agregar salidas con funciones `ui.output_*`()

Para cada salida, defina una función que genere la salida

Llame a los valores de las entradas de la interfaz de usuario con `input.<id>()`

Llame a `App()` para combinar app_ui y servidor en una aplicación interactiva

```
from shiny import App, render, ui
import matplotlib.pyplot as plt
import numpy as np

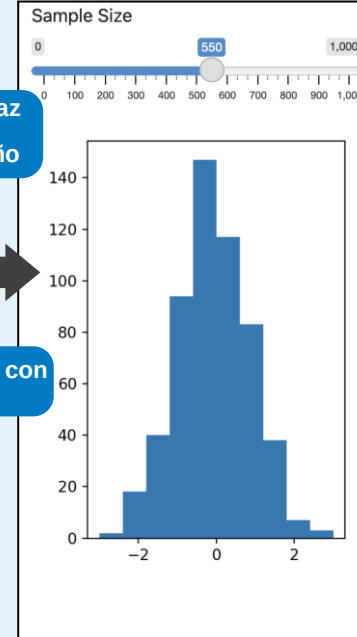
app_ui = ui.page_fluid(
    ui.input_slider(
        "n", "Sample Size", 0, 1000, 20
    ),
    ui.output_plot("dist")
)

def server(input, output, session):
    @render.plot
    def dist():
        x = np.random.randn(input.n())
        plt.hist(x, range=[-3, 3])

app = App(app_ui, server)
```

Diseño de la interfaz de usuario con funciones de diseño

Especifique el tipo de salida con un `@render.decorador`



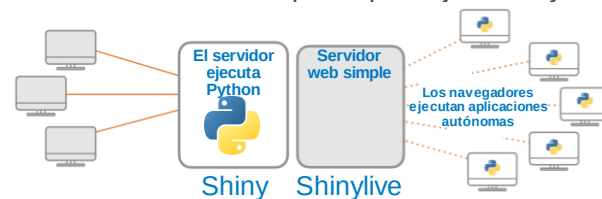
Compartir

Comparte tu aplicación de tres maneras:

1. **Alójala en shinyapps.io**, un servicio basado en la nube de Posit. Para implementar aplicaciones Shiny:
 - Crea una cuenta profesional gratis en shinyapps.io
 - Usar el paquete `reconnect-python` para publicar con `rsconnect deploy shiny <path to directory>`
2. **Compre Posit Connect**, una plataforma de publicación para R y Python. posit.co/connect
3. **Use opciones de despliegue de código abierto** shiny.posit.co/py/docs/deploy.html

Shinylive

Las aplicaciones Shinylive utilizan WebAssembly para ejecutarse completamente en un navegador, sin necesidad de un servidor especial para ejecutar Python.



- Edite y/o aloje aplicaciones Shinylive at shinylive.io
- Crear una versión de Shinylive de una aplicación para implementarla con `shinylive export myapp site`
A continuación, despliegue en un sitio de alojamiento como Github o Netlify
- Incruste aplicaciones Shinylive en sitios, blogs, etc. de Quarto.

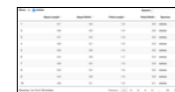
```
filters:
- shinylive

An embedded Shinylive app:
{shinylive-python}
# standalone: true
# [App.py code here...]
```

Para incrustar una aplicación Shinylive en un documento de Quarto, incluya la sintaxis en negrita.

Salidas

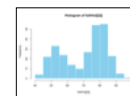
Haga coincidir las funciones `ui.output_*` con los decoradores `@render.*` para vincular la salida de Python a la interfaz de usuario.



`ui.output_data_frame(id)`
`@render.data_frame`



`ui.output_image(id, width, height, click, dblclick, hover, brush, inline)`
`@render.image`



`ui.output_plot(id, width, height, click, dblclick, hover, brush, inline)`
`@render.plot`

Height	Weight	Age	Gender
1.60	55.0	25	Female
1.70	60.0	30	Male
1.80	70.0	35	Female
1.90	80.0	40	Male
2.00	90.0	45	Female

`ui.output_table(id)`
`@render.table`

foo

`ui.output_text_verbatim(id, ...)`
`ui.output_text(id, container, inline)`
`@render.text`



`ui.output_ui(id, inline, container, ...)`
`ui.output_html(id, inline, container, ...)`
`@render.ui`

Download

`ui.download_button(id, label, icon, ...)`
`@session.download`

Entradas

Use una función `ui.` para crear un widget de entrada que guarde un valor como `<id>`. Los valores de entrada son reactivos y deben llamarse como `<id>()`.

Action

`ui.input_action_button(id, label, icon, width, ...)`

Link

`ui.input_action_link(id, label, icon, ...)`

☒ Check me

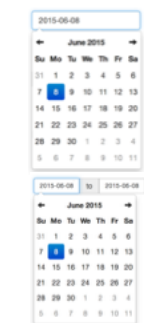
`ui.input_checkbox(id, label, value, width)`

☒ Choice 1

☒ Choice 2

☐ Choice 3

`ui.input_checkbox_group(id, label, choices, selected, inline, width)`



`ui.input_date(id, label, value, min, max, format, startview, weekstart, language, width, autoclose, datesdisabled, daysofweekdisabled)`

Choose File

`ui.input_date_range(id, label, start, end, min, max, format, startview, weekstart, language, separator, width, autoclose)`

1

`ui.input_file(id, label, multiple, accept, width, buttonLabel, placeholder, capture)`

`ui.input_numeric(id, label, value, min, max, step, width)`

☒ Choice A

☐ Choice B

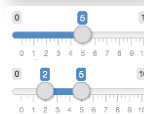
☐ Choice C

`ui.input_password(id, label, value, width, placeholder)`

Choice 1

Choice 2

`ui.input_radio_buttons(id, label, choices, selected, inline, width)`



`ui.input_select(id, label, choices, selected, multiple, selectize, width, size)`
También `ui.input_selectize()`

☐

`ui.input_slider(id, label, min, max, value, step, ticks, animate, width, sep, pre, post, timeFormat, timezone, dragRange)`

Enter text

`ui.input_switch(id, label, value, width)`

`ui.input_text(id, label, value, width, placeholder, autocomplete, spellcheck)`
También `ui.input_text_area()`

